

# Advancing Immersive Content Delivery with Dynamic 3D Gaussian Splatting

Nan Wu\*  
George Mason University  
Fairfax, VA, USA  
nwu5@gmu.edu

Weikai Lin\*  
University of Rochester  
Rochester, NY, USA  
wlin33@ur.rochester.edu

Ruizhi Cheng\*  
George Mason University  
Fairfax, VA, USA  
rcheng4@gmu.edu

Bo Chen  
University of Illinois at  
Urbana-Champaign  
Urbana, IL, USA  
boc2@illinois.edu

Yuhao Zhu  
University of Rochester  
Rochester, NY, USA  
yzhu@rochester.edu

Klara Nahrstedt  
University of Illinois at  
Urbana-Champaign  
Urbana, IL, USA  
klara@illinois.edu

Bo Han  
George Mason University  
Fairfax, VA, USA  
bohan@gmu.edu

## Abstract

Delivering interactive immersive content is essential for emerging augmented, virtual, and mixed reality (AR/VR/MR) applications. However, conventional 3D content representations, such as point clouds and meshes, struggle to capture highly realistic details. While neural radiance fields (NeRF) can generate high-quality content, it encounters issues with real-time rendering on mobile devices. The recent advent of 3D Gaussian splatting (3DGS) offers a promising alternative that enables photo-realistic rendering with lower computational demands than NeRF. Nevertheless, streaming 3DGS-based immersive content to mobile headsets faces significant challenges, including slow computation on mobile devices, long training time for live streaming, and high bandwidth consumption. To tackle these issues, we present a comprehensive research agenda to advance immersive content delivery by leveraging dynamic 3DGS and exploring practical solutions for both video-on-demand and live video streaming services. Our preliminary results demonstrate the potential of our proposed schemes, laying the groundwork for future research.

## CCS Concepts

• **Information systems** → **Multimedia streaming**; • **Computing methodologies** → **Mixed / augmented reality**.

## Keywords

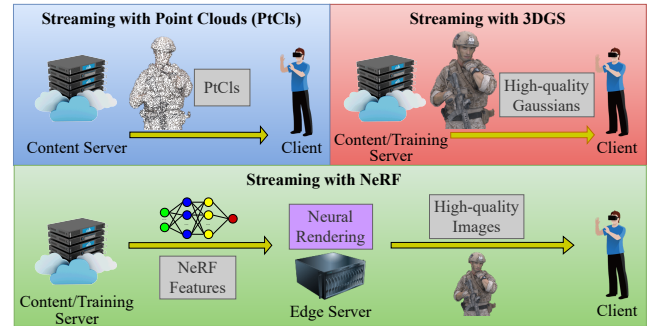
Volumetric Video Streaming and 3D Gaussian Splatting

### ACM Reference Format:

Nan Wu\*, Weikai Lin\*, Ruizhi Cheng\*, Bo Chen, Yuhao Zhu, Klara Nahrstedt, and Bo Han. 2025. Advancing Immersive Content Delivery with Dynamic 3D Gaussian Splatting. In *The 26th International Workshop on Mobile Computing Systems and Applications (HOTMOBILE '25)*, February 26–27, 2025, La Quinta, CA, USA. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3708468.3711886>



This work is licensed under a Creative Commons Attribution 4.0 International License. *HOTMOBILE '25*, February 26–27, 2025, La Quinta, CA, USA  
© 2025 Copyright held by the owner/author(s).  
ACM ISBN 979-8-4007-1403-0/25/02  
<https://doi.org/10.1145/3708468.3711886>



**Figure 1: Comparison of video-on-demand streaming pipelines for point cloud, NeRF, and 3DGS. In this paper, we aim to deliver 3DGS-based immersive content to mobile headsets without involving edge offloading. Servers will be used for model training and storage of content.**

## 1 Introduction

Immersive content delivery enables interactive user experiences with six degrees of freedom (6DoF) motion, essential for natural user interaction in emerging augmented, virtual, and mixed reality (AR/VR/MR) applications. Existing systems typically leverage content representations with explicit data structures such as point clouds [10, 34] and meshes [12]. However, due to their discrete nature, they often fail to accurately capture dynamic elements [21] and realistic lighting [30]. Recently, neural radiance fields (NeRF) [23] has been proposed as an alternative [19], given its higher visual quality than traditional representations. Nonetheless, because of its computation-intensive neural rendering, NeRF requires high-end GPUs to achieve 30 frames per second (FPS) for streaming.

In this paper, we explore 3D Gaussian splatting (3DGS) [13], which offers similar visual quality to NeRF but enables faster rendering. 3DGS [13] represents immersive content with a probabilistic distribution of 3D points, allowing for efficient encoding of geometric details, lighting, and shading. By *combining explicit point representation and differentiable splatting* that empowers gradient-based refinement, 3DGS facilitates not only fast but also photo-realistic

\*These authors contributed equally to this work.

rendering, making it ideal for delivering immersive content to mobile headsets with limited resources (e.g., Microsoft HoloLens 2 [1]). Leveraging the fast rendering capability of 3DGS, we can now ambitiously aim for 90 FPS rendering, instead of 30 FPS targeted by previous studies [6, 10, 34]. Achieving 90 FPS rendering is also required by modern mobile headsets, such as Apple Vision Pro [7].

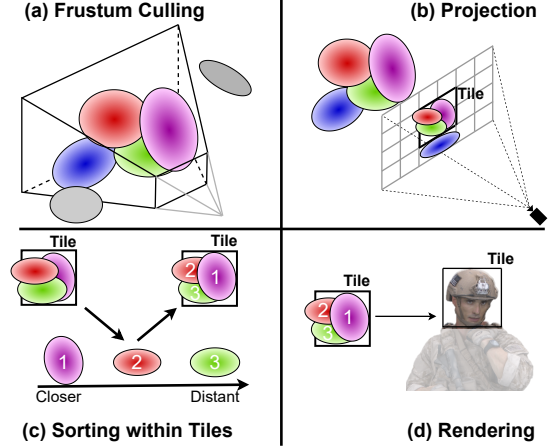
Figure 1 compares the video-on-demand (VOD) streaming with point clouds, NeRF, and 3DGS as immersive content representations. NeRF requires an edge server to perform computationally intensive neural rendering, leading to content drift due to potentially inaccurate prediction of to-be-rendered viewport content [35] that is caused by computation latency and network delay between the server and client [34]. In contrast, 3DGS benefits from explicit content representations that can likely be rendered in real time on mobile devices. Motivated by the attractive features of 3DGS, we present a holistic research agenda to make it practical for immersive content delivery by boosting network and computation efficiency and enhancing the quality of experience (QoE). Our work delves into the research challenges and possible solutions for both VOD and live video streaming (e.g., immersive telepresence [6]) services.

Despite extensive research on extending 3DGS for dynamic scenes [16, 22, 33], the design of 3DGS-based immersive content delivery still poses the following challenges: (1) hardware underutilization, which makes real-time rendering even more challenging, for example,  $\sim 20$  FPS on NVIDIA Jetson Xavier (§3); (2) the absence of a generic, high-quality model capable of handling novel users or appearances (e.g., varying clothing styles); and (3) high bandwidth consumption of streaming immersive content, for example,  $>100$  Mbps streaming at 30 FPS (§3). To tackle these challenges, we propose an agenda guided by the following principles.

- Streaming dynamic VOD 3DGS content to mobile headsets still poses computational challenges due to real-time requirements. Inspired by existing efficient 3DGS implementations [2, 8], we propose two key strategies to alleviate computational pressure and accelerate 3DGS on mobile devices by better utilizing hardware resources. First, we introduce a hardware mapping strategy to optimize the mapping of 3DGS rendering operations to heterogeneous resources on mobile systems on a chip (SoC) [26], improving resource utilization. Second, we propose finer-grained scheduling that addresses the workload imbalance in 3DGS rendering, which arises from variations in workload within tiles (e.g.,  $16 \times 16$  pixels), leading to resource underutilization.

- Compared with VOD services, immersive telepresence that delivers live content enables more compelling and interactive use cases [6]. It requires a generic solution capable of modeling diverse human body movements and appearances in real time and with high visual quality, which, however, existing approaches [11, 17] fail to achieve. In response to these challenges, we propose a lightweight and generic model that directly predicts Gaussian parameters from multi-view input images. Furthermore, benefiting from our proposed model, we optimize both computation and communication overhead by exploiting inter-frame similarity.

We implement several key designs to demonstrate the feasibility of our proposed schemes for 3DGS-based immersive content delivery. Our initial findings highlight the promise of our proposed approaches, setting a strong foundation for future exploration.



**Figure 2: The 3DGS rendering pipeline.** The scene is represented by discrete points, each modeled as a 3D Gaussian distribution. Points outside the camera’s view are discarded through frustum culling (a), while the remaining points are projected onto the image plane and grouped into tiles (b). Within each tile, the points are sorted by depth (c), and the color of each pixel is computed by combining the color the Gaussian points intersecting that pixel (d).

## 2 Background

**Explicit Immersive Content Representations.** Point clouds [10] and meshes [12] are widely adopted representations for immersive content. A point cloud consists of 3D points with attributes, which is effective for non-manifold structures, while its discrete nature can lead to artifacts such as missing pixels [28]. A mesh describes immersive content with triangles, facilitating efficient rendering with lightweight rasterization [20]. However, it struggles to model topological changes due to the dependence of triangles [21]. More importantly, it is challenging for point clouds and meshes to model occlusions and optical effects (e.g., refraction and reflection) [30], resulting in poor visual quality.

**Immersive Content with Implicit Neural Representations.** Recent advancements in immersive content representations have introduced neural networks for photo-realistic rendering. Among them, NeRF [23] leverages multilayer perceptrons (MLPs) to generate implicit 3D representations, enabling highly photorealistic scene rendering by modeling complex lighting interactions in environments. NeRF [23] employs ray marching for rendering, where multiple points along each ray are sampled and processed through a neural network to determine the color of each pixel, resulting in a slow rendering process.

**Immersive Content with Explicit Neural Representations.** 3DGS introduces an explicit neural representation using discrete 3D Gaussian points. These points are parameterized with position, orientation, scale, opacity, and color, enabling detailed and accurate modeling of the scene.

Figure 2 illustrates the rendering pipeline of 3DGS. It begins with frustum culling (step a), where Gaussian points outside the camera’s view are discarded to reduce computational overhead.

The remaining points are projected onto the image plane (step b) and grouped into tiles based on their spatial overlap. This tiling facilitates the efficient processing of Gaussian points in the subsequent steps. Within each tile, the Gaussian points are sorted by depth (step c) to enable accurate handling of occlusions and transparency. In the final stage, the pipeline calculates the contribution of each Gaussian point to the color of the pixels for rendering (step d). For each pixel, intersecting Gaussian points are processed in depth order, blending their contributions to compute the final color. The color of a pixel  $C$  is determined using the volume rendering equation  $C = \sum_{i=0}^{N-1} \alpha_i c_i \prod_{j=1}^{i-1} (1 - \alpha_j)$ , where  $\alpha_i$  and  $c_i$  represent the opacity and color of the  $i$ -th Gaussian point, respectively. Specifically, the color  $c_i$  is computed using 48 spherical harmonic (SH) coefficients, which allows fast calculation of color under varying view directions. By avoiding computationally expensive operations such as ray marching used in NeRF, 3DGS achieves fast rendering.

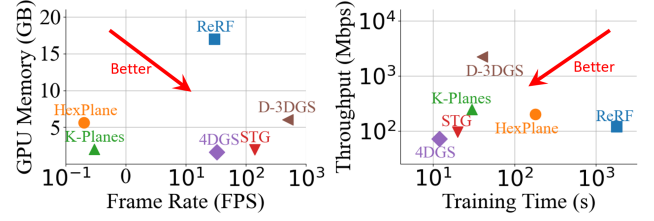
**Hardware-accelerated NeRF & 3DGS.** Hardware accelerations have been proposed to enable real-time mobile inference. In recent years, various architectural supports for NeRF have emerged [14]. Others have explored how to accelerate 3DGS on existing commercial hardware devices, including using the graphics unit to accelerate 3DGS rasterization [4], high-performance CUDA implementations [8], and rewriting 3DGS rendering into a ray-tracing format to leverage ray-tracing units for acceleration [24]. Even with prior acceleration efforts, streaming 3DGS-based content requires accelerating dynamic 3DGS [16, 29, 33], introducing new operations and unique challenges.

### 3 Immersive Content: NeRF vs. 3DGS

This paper focuses on leveraging neural representations for streaming dynamic scenes, which involve moving content or lighting changes over time. To effectively stream such scenes, updated parameters for each frame, including content positions, shapes, and appearance attributes (e.g., colors), must be transmitted to ensure that the rendered output accurately reflects the motion and changes.

**Dynamic NeRF.** For dynamic scenes, recent advances leverage shallow multilayer perception (MLP) networks with NeRF features saved in sparse geometric representations to accelerate rendering and capture extensive dynamics [31]. The state-of-the-art streamable NeRF, ReRF [31], effectively handles long-sequence videos by capturing residual information between adjacent frames in the spatial-temporal feature space. Nevertheless, ReRF still relies on computation-intensive ray marching for rendering and consumes significant GPU memory, making it less practical for resource-constrained mobile devices.

**Dynamic 3DGS** extends the original method by tracking and updating the parameters (e.g., position and rotation) of each Gaussian point over time. For example, Spacetime Gaussian (STG) [16] introduces time-dependent parameters within each Gaussian point to capture complex content changes. Additionally, since each Gaussian point is represented independently, STG is resilient to packet loss, empowering robust streaming in unstable network environments. These designs make STG a state-of-the-art solution for streamable 3DGS. However, streaming numerous Gaussian points still leads to high bandwidth consumption.



**Figure 3: Comparison of variations/improvements of dynamic NeRF and 3DGS on the *cook\_spinach* dataset [15].**

**Comparison of NeRF and 3DGS.** We compare recently proposed dynamic NeRF and 3DGS schemes on the *cook\_spinach* dataset [15] with an NVIDIA RTX 4090 GPU. Among them, HexPlane [5], K-Planes [9], and ReRF [31] are for NeRF, and 4DGS [33], STG [16], and D-3DGS [22] are for 3DGS. The rendering quality of all methods is comparable. Thus, we mainly focus on the GPU memory usage, rendering frame rate, throughput (at 30 FPS), and per-frame training time when leveraging them for streaming. As shown in Figure 3, NeRF-based methods usually require higher GPU memory and have low rendering frame rates. Only ReRF [31] can achieve  $\sim 30$  FPS rendering, which is comparable with 4DGS, the slowest method for 3DGS-based streaming while consuming  $>8\times$  GPU memory. Moreover, 3DGS-based methods usually require less training time. The recent advent of STG [16] efficiently utilizes the temporal correlation of Gaussian points between frames, reducing the throughput and keeping the training time  $<20$  seconds for each frame.

Despite recent advancements in immersive content delivery and 3DGS, several challenges persist in leveraging dynamic 3DGS for immersive video streaming. First, the long rendering time remains challenging, especially when computational resources are limited. For example, STG achieves only  $\sim 20$  FPS on NVIDIA Jetson Xavier NX (with GPU performance comparable [3] to that of Meta Quest 2 headset), which falls short of delivering the smooth performance required for real-time applications. Additionally, the training time for dynamic 3DGS can take several seconds per frame, making it impractical for immersive telepresence. Furthermore, the state-of-the-art methods may consume  $>100$  Mbps for streaming, which many current Internet connections may not adequately support [6].

## 4 Proposed Research Agenda

### 4.1 Accelerating 3DGS for Streaming

**Problem and Challenges.** Delivering dynamic 3DGS for immersive experiences requires rendering of each frame to be completed within a short time period (e.g.,  $<11$  ms for  $>90$  FPS). However, dynamic 3DGS achieves only  $\sim 20$  FPS on NVIDIA Jetson Xavier. Our profiling using STG [16] with the *cook\_spinach* dataset [15] (Figure 4) shows that rendering and MLP decoding are the main bottlenecks, with projection and sorting also contributing. We find that these inefficiencies primarily arise from resource underutilization. First, 3DGS variants are commonly implemented on GPU compute units due to their flexibility for general-purpose operations. However, modern SoCs offer various heterogeneous resources, such as neural processing units (NPUs) and graphic units, which often remain idle during 3DGS rendering. Furthermore, the 3DGS implementation is inherently inefficient even within compute units,



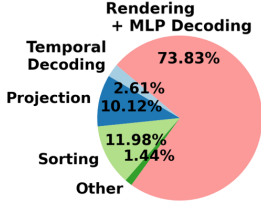


Figure 4: Execution time breakdown of rendering dynamic 3DGS on the *cook\_spinach* dataset [15].



Figure 5: Workload imbalance of dynamic 3DGS (heatmap showing the number of Gaussian points).

leading to resource underutilization. For example, 3DGS divides pixels into tiles, processing each tile with a warp [13]. Since the workload within a warp must start and finish together, if certain pixels within a tile finish earlier, they cannot exit the warp, leading to idle resource wastage.

**Proposed Research.** To mitigate hardware underutilization and reduce computational pressure on mobile devices, we propose algorithm-hardware mapping and finer-grained scheduling.

**Efficient Rendering with Heterogeneous Units via Algorithm-hardware Mapping.** 3DGS [13] relies solely on GPU compute units, underutilizing heterogeneous modern mobile SoC resources such as NPU and graphic units. To address this, we can leverage smarter algorithm-to-hardware mapping. For example, the MLP decoding commonly used in dynamic 3DGSs [16, 29, 33] can be offloaded to NPUs. Similarly, the rendering stage of 3DGS, which closely resembles traditional rasterization, can be mapped to graphic units, eliminating the need to occupy compute shaders [2]. However, achieving optimal mapping is non-trivial. First, operations in 3DGS do not always align with common SoC hardware. For example, to leverage ray-tracing units in SoC, we need to reformulate 3DGS in a ray-tracing format [24]. Second, indiscriminate offloading tasks may turn out to leave compute units idle and underutilized, making moving certain tasks back a better choice instead.

We propose two research solutions. First is *hardware-aware 3DGS*, where we re-design 3DGS to better align with hardware units, enabling more efficient offloading. For example, existing graphic units are mostly designed for triangular meshes, whereas 3DGS projects points into ellipses on the image. This mismatch makes hardware acceleration less efficient. If the shape of 3DGS points could be modeled as triangular meshes, it would better utilize existing hardware. However, changing the original shape could introduce quality loss, which requires us to evaluate the trade-offs between efficiency and visual fidelity carefully. Second is *optimization-based mapping*, which abstracts the mapping process as a constrained optimization problem aimed at maximizing throughput while respecting existing heterogeneous hardware resource constraints. Formally, given dynamic 3DGS rendering stages  $\mathcal{S}$ , different hardware  $\mathcal{H}$ , the optimization can be written as:

$$\min \sum_{s \in \mathcal{S}} T_s \quad \text{subject to: } U(h, t) \leq 1, \forall h \in \mathcal{H}, t \in \mathcal{T}. \quad (1)$$

Here,  $\mathcal{T}$  is the set of unit time steps, and  $T_s$  denotes the time consumed by each stage  $s \in \mathcal{S}$ , and  $U(h, t)$  represents the utilization of hardware  $h \in \mathcal{H}$  at operation time  $t \in \mathcal{T}$ . The objective is to

minimize the total time consumed across all stages while ensuring that the utilization of any hardware does not exceed 1 at any given time. This approach achieves a more balanced workload distribution across units, boosting FPS compared to greedy offloading.

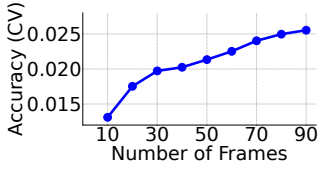
**Efficient Finer-grained Scheduling for Enhanced Resource Utilization.** 3DGS rendering faces inefficiencies due to workload imbalances within tiles (e.g.,  $16 \times 16$  pixels). As shown in Figure 5, since all pixels within a tile are executed by the same warp and must finish simultaneously, intra-tile imbalances lead to idle compute units as faster pixels are forced to wait for slower ones to complete. To address this, we propose a finer-grained scheduling strategy. Instead of naively using grid-based tile separation, we can split pixel workloads into more balanced groups, improving hardware utilization. However, pixel workloads cannot be determined prior to rendering, and scheduling incurs overhead. Fortunately, in dynamic 3DGS, the workload distribution of adjacent frames tends to be similar. This allows us to use pixel workloads from past frames to schedule the current frame. Specifically, we record the moving average of pixel workloads from previous frames and use this average as the predicted pixel workload for the current frame during scheduling. Furthermore, the scheduling overhead can be amortized by reusing the scheduling results across multiple frames. To be specific, we can define a window size  $N$ , where only the first frame within each window is scheduled, and the scheduling result is reused for the subsequent  $N - 1$  frames. This approach effectively reduces scheduling workload by  $N \times$ .

## 4.2 Boosting Immersive Telepresence

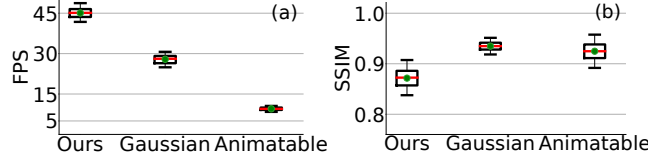
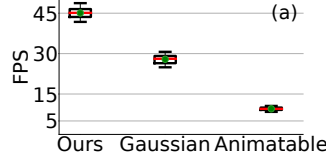
**Problem and Challenges.** Compared with VOD service, immersive telepresence that delivers live content offers a wider array of compelling applications [6]. We focus on users as the main subject of immersive content, as they are the key component in telepresence [6]. Utilizing 3DGS for telepresence presents the following challenges. First, as the content of future frames is unknown, we should dynamically generate Gaussian points that accurately represent diverse body poses in real time. Also, the solution should be generic and not require re-training for new users or appearances (e.g., different clothing styles), as such process is typically time-consuming (Figure 3). Second, maintaining high visual quality requires generating and transmitting a dense set of Gaussian points, which can result in substantial computational overhead and bandwidth consumption.

**Proposed Research.** We design an efficient and generic model for generating 3D Gaussian parameters from multi-view images and optimize both computational and communication overhead by leveraging inter-frame similarity.

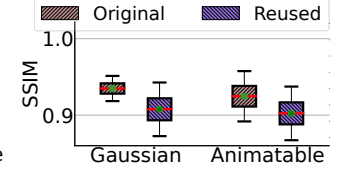
**Efficient and Generic Human Body Modeling.** Existing solutions [11, 17] for modeling human body movements with 3DGS primarily rely on parametric models, such as SMPL-X [25]. However, estimating SMPL-X parameters is computationally intensive. Our experiments reveal that the state-of-the-art SMPL-X estimation solution [6] operates  $<10$  FPS on NVIDIA Jetson Xavier NX. Additionally, generating 3D Gaussian points from the output mesh of SMPL-X involves complex 3D-to-3D transformations, which introduces further computational overhead (§5). Moreover, we find



**Figure 6: Coefficient of Variation (CV) of pixel workloads across frames (lower CV indicates better accuracy).**



**Figure 7: Comparison of (a) FPS and (b) SSIM of our proposed solution, GaussianAvatar [11], and Animatable Gaussians [17]. The experiments are conducted on an edge server equipped with an NVIDIA RTX 4090 GPU.**



**Figure 8: SSIM of GaussianAvatar [11] and Animatable Gaussians [17] when reusing SMPL-X parameters [25].**

that existing solutions lack generalizability to novel human subjects and appearances.

To address these limitations, we propose a regression-based approach that directly predicts 2D Gaussian parameters from source images without relying on parametric models. Specifically, our approach takes multi-view RGB images as input and uses a pixel-wise regression framework to predict 2D Gaussian parameters. They are then lifted to 3D using depth maps, which can be obtained, for example, from RGB-D sensors. As the training/inference process involves 2D-to-2D operations, instead of complex 3D-to-3D transformations, it is inherently more efficient than existing solutions [11, 17]. Furthermore, the pixel-wise regression approach offers strong generalizability when trained on large-scale datasets of 3D human body scans. This generalization capability arises because the model learns directly from source images, allowing it to simultaneously capture both appearance details (*e.g.*, textures and clothing) and body structure information (*e.g.*, pose and shape).

**Optimizing Computation and Communication Overhead with Inter-frame Similarity.** To achieve high visual fidelity, we should create dense Gaussian parameters, which may lead to substantial computation overhead, even with the optimizations provided by our 2D-based inference approach. Additionally, the inference should be performed on the sender side to balance the computational load between the sender and receiver, as the receiver must render dense 3D Gaussian parameters. Hence, the transmission data between the sender and receiver consists of these dense parameters, which results in significant bandwidth consumption.

Motivated by the observation that, during body movements, only certain regions of the body may exhibit significant changes in some frames compared to previous ones, we propose a strategy to minimize computation and communication overhead by leveraging inter-frame similarity. Specifically, we perform regression on only regions that exhibit notable changes and transmit the updated parameters to the receiver. Different from previous work that pre-defines a fixed region to determine updates [6], our approach enables a more fine-grained determination of changes, thanks to our pixel-wise regression model. The main challenge is to set an appropriate threshold for pixel similarity, as it involves a trade-off between computation, communication, and visual quality. To address it, we propose conducting a large-scale user study to evaluate the effects of different thresholds under various network conditions. We will collect key metrics such as FPS, bandwidth consumption, structural similarity index measure (SSIM) [32], and mean opinion score (MOS) to determine the optimal thresholds.

## 5 Preliminary Results

**Accelerating Dynamic 3DGS Rendering.** We design experiments to validate the feasibility of ideas in Sec 4.1. We use the Jetson Xavier, applying 4DGS [33] on the *cook\_spinach* dataset [15]. To validate that heterogeneous SoC hardware can accelerate rendering, we use FastGS [2] for offloading rasterization operations to graphic units and show results across different resolutions. Our experiments show that speedup ranges from 2.0 ~ 5.5 $\times$  and is particularly significant at higher resolutions. The improvements are 2.0 $\times$  for the resolution of  $1352 \times 1014$ , 3.6 $\times$  for  $2704 \times 2028$ , 4.5 $\times$  for  $4056 \times 3042$ , 5.1 $\times$  for  $5408 \times 4056$ , and 5.5 $\times$  for  $6760 \times 5070$ . These highlight the potential of using heterogeneous units to accelerate dynamic 3DGS.

We further validate the feasibility of our efficient finer-grained scheduling by analyzing pixel workload variations across frames, which supports our idea of leveraging past frame workloads to guide scheduling. The variation is quantified using the coefficient of variation (CV), defined as the standard deviation divided by the mean, where a lower CV indicates better accuracy. We vary the number of frames and record the average pixel workload variation in Figure 6. The results indicate that the CV remains consistently low ( $CV < 0.02$ ) for up to 40 frames, emphasizing the robustness and potential of our approach.

**Efficient Human Body Modeling.** We have partially implemented our proposed design, which directly regresses Gaussian parameters from source images (§4.2) without relying on parametric models such as SMPL-X [25]. We utilize the dataset from MagicStream [6], which provides RGB and depth images of full human bodies. Our approach adopts an encoder-decoder architecture, leveraging RAFT-Stereo [18] to encode 2D source images into features and U-Net [27] to decode these features into 2D Gaussian parameters. The predicted parameters are lifted into 3D using the depth images and rendered through the 3DGS rendering pipeline. All experiments are conducted at a rendering resolution of  $1280 \times 720$ , consistent with the setup used in MagicStream [6], on an edge server equipped with an NVIDIA RTX 4090 GPU and an AMD Ryzen 9 7900X CPU.

Figure 7 compares our proposed model against two state-of-the-art SMPL-X-based solutions, GaussianAvatar [11] and Animatable Gaussians [17], in terms of FPS and SSIM [32]. Our model achieves 45 FPS on average, exceeding the 30 FPS target established by prior efforts [6] and outperforming GaussianAvatar (28 FPS) and Animatable Gaussians (10 FPS). Also, our model eliminates the need to eliminate SMPL-X parameters, which can maintain higher FPS and reduce end-to-end latency compared to SMPL-X-based approaches. Nevertheless, our proposed model still remains a gap to the 90

FPS required for modern headsets, such as Apple Vision Pro [7], highlighting the need for further optimization.

In terms of visual quality, our model achieves an average SSIM of 0.87, which is lower than SMPL-X-based solutions, for example, 0.93 achieved by GaussianAvatar. This discrepancy can be attributed to the strong geometric and anthropometric priors provided by SMPL-X, which effectively constrain the solution space to plausible human shapes and poses. These structured priors are difficult to replicate in a direct regression, highlighting a trade-off between FPS/latency and visual quality that necessitates further investigation.

**Inter-frame Similarity.** We next investigate the potential of leveraging inter-frame similarity to enhance immersive telepresence (§4.2). We conduct experiments with GaussianAvatar [11] and Animatable Gaussians [17]. To demonstrate inter-frame similarity, we design an experiment where we reuse SMPL-X parameters [25] from previous frames for odd-numbered frames (e.g., 1, 3, and 5), and calculate the SSIM [32] to assess the visual quality for these frames. Figure 8 presents the SSIM for both examined methods, with and without SMPL-X parameter reuse. Although reusing SMPL-X parameters results in a slight decrease in SSIM, it still remains >0.9 on average, indicating good visual quality [6]. These results suggest that significant inter-frame similarity exists in human body movements, validating the feasibility of our proposed approach.

## 6 Conclusion

In this paper, we presented a research agenda that addresses the key challenges of streaming dynamic 3DGS content for immersive experiences. To overcome the computational challenges on mobile devices, we introduced hardware-aware strategies, leveraging modern GPU and SoC architectures to accelerate 3DGS rendering. For immersive telepresence, we proposed a lightweight, generic human body model to predict Gaussian parameters from multi-view images and optimize computational and communication efficiency. We aspire for our research to spark new investigations into 3DGS-based immersive content delivery.

## Acknowledgment

We thank the anonymous reviewers and our shepherd Robert LiKamWa for their insightful feedback. This work was partially supported by the National Science Foundation under Grants CNS-2402992, CNS-2225860, CNS-2106592, and OAC-1835834.

## References

- [1] 2019. Microsoft HoloLens 2. <https://www.microsoft.com/en-us/hololens>. [accessed on 13-Jan-2025].
- [2] 2024. Fast Gaussian Rasterization. <https://github.com/dendenxu/fast-gaussian-rasterization>. [accessed on 13-Jan-2025].
- [3] 2024. Qualcomm Adreno 650 vs Nvidia Jetson Xavier NX GPU. <https://bit.ly/42srjGz>. [accessed on 13-Jan-2025].
- [4] 2024. Vulkan-based Gaussian Splatting Viewer and Python binding. <https://github.com/jaesung-cs/vkgs>. [accessed on 13-Jan-2025].
- [5] Ang Cao and Justin Johnson. 2023. Hexplane: A Fast Representation for Dynamic Scenes. In *Proceedings of CVPR*.
- [6] Ruizhi Cheng, Nan Wu, Vu Le, Eugene Chai, Matteo Varvello, and Bo Han. 2024. MagicStream: Bandwidth-conserving Immersive Telepresence via Semantic Communication. In *Proceedings of ACM SenSys*.
- [7] Ruizhi Cheng, Nan Wu, Matteo Varvello, Eugene Chai, Songqing Chen, and Bo Han. 2024. A First Look at Immersive Telepresence on Apple Vision Pro. In *Proceedings of ACM IMC*.
- [8] Guofeng Feng, Siyan Chen, Rong Fu, Zimu Liao, et al. 2024. FlashGS: Efficient 3D Gaussian Splatting for Large-scale and High-resolution Rendering. [www.arxiv.org/abs/2408.07967](https://arxiv.org/abs/2408.07967). [accessed on 13-Jan-2025].
- [9] Sara Fridovich-Keil, Giacomo Meanti, Frederik Rahbæk Warburg, Benjamin Recht, and Angjoo Kanazawa. 2023. K-Planes: Explicit Radiance Fields in Space, Time, and Appearance. In *Proceedings of CVPR*.
- [10] Bo Han, Yu Liu, and Feng Qian. 2020. ViVo: Visibility-aware Mobile Volumetric Video Streaming. In *Proceedings of ACM MobiCom*.
- [11] Liangxiao Hu, Hongwen Zhang, Yuxiang Zhang, et al. 2024. GaussianAvatar: Towards Realistic Human Avatar Modeling from a Single Video via Animatable 3D Gaussians. In *Proceedings of CVPR*.
- [12] Tao Jin, Mallesham Dasa, Connor Smith, Kittipat Apicharttrisor, Srinivasan Seshan, and Anthony Rowe. 2024. MeshReduce: Scalable and Bandwidth Efficient 3D Scene Capture. In *Proceedings of IEEE VR*.
- [13] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 2023. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. *ACM Transactions on Graphics (ToG)* 42, 4 (2023), 1–14.
- [14] Junseo Lee, Kwansook Choi, Jungi Lee, Seokwon Lee, Joonho Whangbo, and Jaewoong Sim. 2023. Neurex: A Case for Neural Rendering Acceleration. In *Proceedings of ISCA*.
- [15] Tianye Li, Mira Slavcheva, Michael Zollhoefer, Simon Green, Christoph Lassner, Changil Kim, Tanner Schmidt, et al. 2022. Neural 3D Video Synthesis from Multi-view Video. In *Proceedings of CVPR*.
- [16] Zhan Li, Zhang Chen, Zhong Li, and Yi Xu. 2024. Spacetime Gaussian Feature Splatting for Real-Time Dynamic View Synthesis. In *Proceedings of IEEE/CVF CVPR*.
- [17] Zhe Li, Zerong Zheng, Lizhen Wang, and Yebin Liu. 2024. Animatable Gaussians: Learning Pose-dependent Gaussian Maps for High-fidelity Human Avatar Modeling. In *Proceedings of CVPR*.
- [18] Lahav Lipson, Zachary Teed, and Jia Deng. 2021. Raft-Stereo: Multilevel Recurrent Field Transforms for Stereo Matching. In *Proceedings of International Conference on 3D Vision (3DV)*.
- [19] Kaiyan Liu, Ruizhi Cheng, Nan Wu, and Bo Han. 2023. Toward Next-generation Volumetric Video Streaming with Neural-based Content Representations. In *Proceedings of ACM ImmerCom*.
- [20] Shichen Liu, Tianye Li, Weikai Chen, and Hao Li. 2019. Soft Rasterizer: A Differentiable Renderer for Image-based 3D Reasoning. In *Proceedings of CVPR*.
- [21] Stephen Lombardi, Tomas Simon, Gabriel Schwartz, Michael Zollhoefer, et al. 2021. Mixture of Volumetric Primitives for Efficient Neural Rendering. *ACM Transactions on Graphics (ToG)* 40, 4 (2021), 1–13.
- [22] Jonathon Luiten, Georgios Kopanas, Bastian Leibe, and Deva Ramanan. 2024. Dynamic 3D Gaussians: Tracking by Persistent Dynamic View Synthesis. In *Proceedings of International Conference on 3D Vision*.
- [23] Ben Mildenhall, Pratul P Srinivasan, Matthew Tancik, Jonathan T Barron, et al. 2021. NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis. *Commun. ACM* 65, 1 (2021), 99–106.
- [24] Nicolas Moenne-Loccoz, Ashkan Mirzaei, Or Perel, Riccardo de Lutio, et al. 2024. 3D Gaussian Ray Tracing: Fast Tracing of Particle Scenes. [www.arxiv.org/abs/2407.07090](https://arxiv.org/abs/2407.07090). [accessed on 13-Jan-2025].
- [25] Georgios Pavlakos, Vasileios Choutas, Nima Ghorbani, Timo Bolkart, Ahmed AA Osman, et al. 2019. Expressive Body Capture: 3D Hands, Face, and Body from a Single Image. In *Proceedings of CVPR*.
- [26] Qualcomm. 2024. Snapdragon XR2 Gen 2 Reference Design Product Brief. <https://bit.ly/3Y2cqug>. [accessed on 13-Jan-2025].
- [27] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. 2015. U-net: Convolutional Networks for Biomedical Image Segmentation. In *Proceedings of International Conference on Medical Image Computing and Computer Assisted Intervention*.
- [28] Hang Su, Varun Jampani, Deqing Sun, Subhransu Maji, Evangelos Kalogerakis, Ming-Hsuan Yang, and Jan Kautz. 2018. Splatnet: Sparse Lattice Networks for Point Cloud Processing. In *Proceedings of CVPR*.
- [29] Jiakai Sun, Han Jiao, Guangyuan Li, Zhanjie Zhang, et al. 2024. 3DGStream: On-the-Fly Training of 3D Gaussians for Efficient Streaming of Photo-Realistic Free-Viewpoint Videos. In *Proceedings of CVPR*.
- [30] Hanchen Wang, Qi Liu, Xiangyu Yue, Joan Lasenby, and Matt J Kusner. 2021. Unsupervised Point Cloud Pre-training via Occlusion Completion. In *Proceedings of ICCV*.
- [31] Liao Wang, Qiang Hu, Qihan He, Ziyu Wang, Jingyi Yu, Tinne Tuytelaars, Lan Xu, and Minye Wu. 2023. Neural Residual Radiance Fields for Streamably Free-Viewpoint Videos. In *Proceedings of CVPR*.
- [32] Zhou Wang, Alan C Bovik, Hamid R Sheikh, and Eero P Simoncelli. 2004. Image Quality Assessment: from Error Visibility to Structural Similarity. *IEEE Transactions on Image Processing* 13, 4 (2004), 600–612.
- [33] Guanjun Wu, Taoran Yi, Jiemin Fang, Lingxi Xie, Xiaopeng Zhang, Wei Wei, Wenyu Liu, Qi Tian, Wang, et al. 2024. 4D Gaussian Splatting for Real-Time Dynamic Scene Rendering. In *Proceedings of CVPR*.
- [34] Nan Wu, Kaiyan Liu, Ruizhi Cheng, Bo Han, and Puqi Zhou. 2024. Theia: Gaze-driven and Perception-aware Volumetric Content Delivery for Mixed Reality Headsets. In *Proceedings of ACM MobiSys*.
- [35] Tan Xu, Bo Han, and Feng Qian. 2019. Analyzing viewport prediction under different VR interactions. In *Proceedings of ACM CoNEXT*.